Introduction of ADO.Net,

It is a module of .Net Framework which is used to establish connection between application and data sources. Data sources can be such as SQL Server and XML. ADO.NET consists of classes that can be used to connect, retrieve, insert and delete data.

Namespaces used in ADO.Net

In any .NET data access page, before you connect to a database, you first have to import all the necessary namespaces that will allow you to work with the objects required. Namespaces used in ADO.Net are:

1. **System.Data**

   It contains the common classes for connecting, fetching data from database. Classes are like as DataTable, DataSet, DataView etc.

2. **System.Data.SqlClient**

   It contains classes for connecting, fetching data from Sql Server database. Classes are like as SqlDataAdapter,SqlDataReader etc.

3. **System.Data.OracleClient**

   It contains classes for connecting, fetching data from Oracle database. Classes are like as OracleDataAdapter,OracleDataReader etc.

4. **System.Data.OleDb**

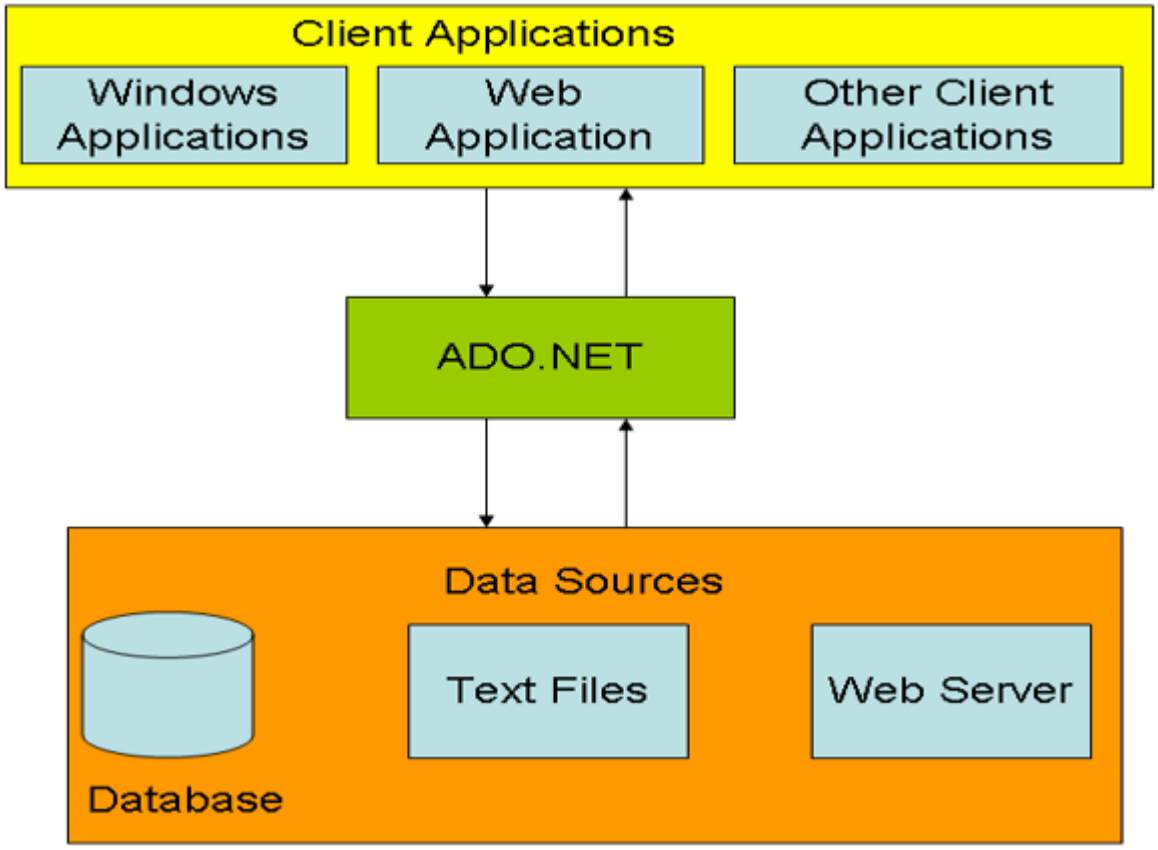
   It contains classes for connecting, fetching data from any database (like msaccess, db2, oracle, sqlserver, mysql). Classes are like as OleDbDataAdapter,OleDbDataReader etc.

5. **System.Data.Odbc**

   It contains classes for connecting, fetching data from any database (like msaccess, db2, oracle, sqlserver, mysql). Classes are like as OdbcDataAdapter,OdbcDataReader etc.

->ADO.net Architecture

It is a model used by the applications to communicate with a database for manipulating data as shown in the following figure.



In the above figure, client application can be Windows application, Web application or any otheroffice applications, mobile applications. These applications need to retrieve and access from various data sources. The sources can be SQL Server, Oracle or OLEDB.

Features in ADO.NET

1. Bulk Copy Operation
2. Batch Update
3. Data Paging
4. Connection Details
5. DataSet.RemotingFormat Property
6. DataTable's Load and Save Methods
7. New Data Controls
8. DbProvidersFactories Class
9. Customized Data Provider
10. DataReader's New Execute Methods

1.DataSet

->DataSet is a disconnected architecture it represents the data in table structure which means the data into rows and columns.

-> Dataset is the local copy of your database which exists in the local system and makes the application execute faster and reliable.

->DataSet works like a real database with an entire set of data which includes the constraints, relationship among tables, and so on. It will be found in the namespace “System. Data”.

DataSet Constructors

Constructor	Description
DataSet()	It is used to initialize a new instance of the DataSet class.
DataSet(String)	It is used to initialize a new instance of a DataSet class with the given name.
DataSet(SerializationInfo, StreamingContext)	It is used to initialize a new instance of a DataSet class that has the given serialization information and context.
DataSet(SerializationInfo, StreamingContext, Boolean)	It is used to initialize a new instance of the DataSet class.

DataSet Properties

Properties	Description
CaseSensitive	It is used to check whether DataTable objects are case-sensitive or not.
DataSetName	It is used to get or set name of the current DataSet.
DefaultViewManager	It is used to get a custom view of the data contained in the DataSet to allow filtering and searching.
HasErrors	It is used to check whether there are errors in any of the DataTable objects within this DataSet.
IsInitialized	It is used to check whether the DataSet is initialized or not.
Locale	It is used to get or set the locale information used to compare strings within the table.
Namespace	It is used to get or set the namespace of the DataSet.
Site	It is used to get or set an ISite for the DataSet.
Tables	It is used to get the collection of tables contained in the DataSet.

DataSet Methods

The following table contains some commonly used methods of DataSet.

Method	Description
BeginInit()	It is used to begin the initialization of a DataSet that is used on a form.
Clear()	It is used to clear the DataSet of any data by removing all rows in all tables.
Clone()	It is used to copy the structure of the DataSet.
Copy()	It is used to copy both the structure and data for this DataSet.
CreateDataReader(DataTable[])	It returns a DataTableReader with one result set per DataTable.

CreateDataReader()	It returns a DataTableReader with one result set per DataTable.
EndInit()	It ends the initialization of a DataSet that is used on a form.
GetXml()	It returns the XML representation of the data stored in the DataSet.
GetXmlSchema()	It returns the XML Schema for the XML representation of the data stored in the DataSet.
Load(IDataReader, LoadOption, DataTable[])	It is used to fill a DataSet with values from a data source using the supplied IDataReader.
Merge(DataSet)	It is used to merge a specified DataSet and its schema into the current DataSet.
Merge(DataTable)	It is used to merge a specified DataTable and its schema into the current DataSet.
ReadXml(XmlReader, XmlReadMode)	It is used to read XML schema and data into the DataSet using the specified XmlReader and XmlReadMode.
Reset()	It is used to clear all tables and removes all relations, foreign constraints, and tables from the DataSet.
WriteXml(XmlWriter, XmlWriteMode)	It is used to write the current data and optionally the schema for the DataSet using the specified XmlWriter and XmlWriteMode.

Example:

```
using System;
using System.Data.SqlClient;
using System.Data;
namespace DataSetExample
{
   public partial class DataSetDemo : System.Web.UI.Page
   {
      protected void Page_Load(object sender, EventArgs e)
      {
         using (SqlConnection con = new SqlConnection("data source=.; database=student; integrated security=SSPI"))
         {
            SqlDataAdapter sde = new SqlDataAdapter("Select * from student", con);
            DataSet ds = new DataSet();
            sde.Fill(ds);
            GridView1.DataSource = ds;
            GridView1.DataBind();
```

```
        }
      }
    }
  }
```

2.DataProvider

->Data provider is used to connect to the database, execute commands and retrieve the record.
->It is lightweight component with better performance.
-> It also allows us to place the data into DataSet to use it further in our application.

.NET Framework data provider	Description
.NET Framework Data Provider for SQL Server	It provides data access for Microsoft SQL Server. It requires the **System.Data.SqlClient** namespace.
.NET Framework Data Provider for OLE DB	It is used to connect with OLE DB. It requires the **System.Data.OleDb** namespace.

.NET Framework Data Providers Objects

Following are the core object of Data Providers.

Object	Description
Connection	It is used to establish a connection to a specific data source.
Command	It is used to execute queries to perform database operations.
DataReader	It is used to read data from data source. The DbDataReader is a base class for all DataReader objects.
DataAdapter	It populates a DataSet and resolves updates with the data source. The base class for all DataAdapter objects is the DbDataAdapter class.

.NET Framework Data Provider for SQL Server

Data provider for SQL Server is a lightweight component. It provides better performance because it directly access SQL Server without any middle connectivity layer. In early versions, it interacts with ODBC layer before connecting to the SQL Server that created performance issues.

The .NET Framework Data Provider for SQL Server classes is located in the **System.Data.SqlClient** namespace. We can include this namespace in our C# application by using the following syntax.

```
using System.Data.SqlClient;
```

This namespace contains the following important classes.

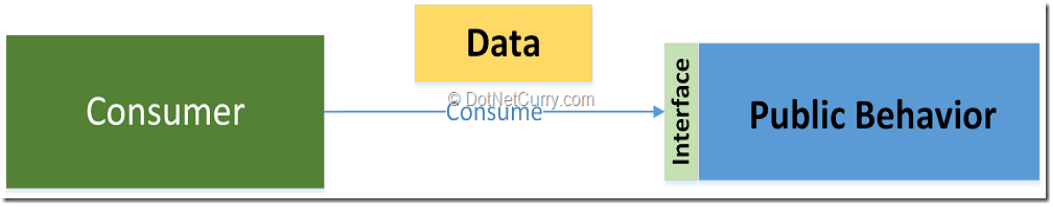
Class	Description
SqlConnection	It is used to create SQL Server connection. This class cannot be inherited.
SqlCommand	It is used to execute database queries. This class cannot be inherited.
SqlDataAdapter	It represents a set of data commands and a database connection that are used to fill the DataSet. This class cannot be inherited.
SqlDataReader	It is used to read rows from a SQL Server database. This class cannot be inherited.
SqlException	This class is used to throw SQL exceptions. It throws an exception when an error is occurred. This class cannot be inherited.

Example

```
// Creating Connection
        con = new SqlConnection("data source=.; database=student; integrated security=SSPI");
        // writing sql query
        SqlCommand cm = new SqlCommand("create table student(id int not null,
        name varchar(100), email varchar(50), join_date date)", con);
        // Opening Connection
        con.Open();
        // Executing the SQL query
        cm.ExecuteNonQuery();
        // Displaying a message
        Console.WriteLine("Table created Successfully");
```

3 DataObject,

When doing behavior-only encapsulation, we separate data and behavior into separate units. Data units contain only data that is accessible directly, and behavior units contain only behavior. Such behavior units are similar to functions and procedures in functional and procedural programming, respectively.



Make your data objects immutable

This means that once a data object is created, its contents will never change.

In C#, we can do this by making all properties read-only and providing a constructor to allow the initialization of these properties like this:

public class TextDocument

```
{
   public TextDocument(string identifier, string content)
   {
      Identifier = identifier;
      Content = content;
   }

   public string Identifier { get; }

   public string Content { get; }
}
```

4.DataAdapter

The DataAdapter works as a
bridge between a DataSet and a data source to retrieve data. DataAdapter is a class that represents a set of SQL commands and a database connection. It can be used to fill the DataSet and update the data source.

DataAdapter Constructors

Constructors	Description
DataAdapter()	It is used to initialize a new instance of a DataAdapter class.
DataAdapter(DataAdapter)	It is used to initializes a new instance of a DataAdapter class from an existing object of the same type.

Methods

Method	Description
CloneInternals()	It is used to create a copy of this instance of DataAdapter.
Dispose(Boolean)	It is used to release the unmanaged resources used by the DataAdapter.
Fill(DataSet)	It is used to add rows in the DataSet to match those in the data source.
FillSchema(DataSet, SchemaType, String, IDataReader)	It is used to add a DataTable to the specified DataSet.
GetFillParameters()	It is used to get the parameters set by the user when executing an SQL SELECT statement.
ResetFillLoadOption()	It is used to reset FillLoadOption to its default state.
ShouldSerializeAcceptChangesDuringFill()	It determines whether the AcceptChangesDuringFill property should be persisted or not.
ShouldSerializeFillLoadOption()	It determines whether the FillLoadOption property should be

	persisted or not.
ShouldSerializeTableMappings()	It determines whether one or more DataTableMapping objects exist or not.
Update(DataSet)	It is used to call the respective INSERT, UPDATE, or DELETE statements.

Example

```
using System;
using System.Data.SqlClient;
using System.Data;
namespace DataSetExample
{
   public partial class DataSetDemo : System.Web.UI.Page
   {
      protected void Page_Load(object sender, EventArgs e)
      {
         using (SqlConnection con = new SqlConnection("data source=.; database=student; integrated security=SSPI"))
         {
            SqlDataAdapter sde = new SqlDataAdapter("Select * from student", con);
            DataSet ds = new DataSet();
            sde.Fill(ds);
            GridView1.DataSource = ds;
            GridView1.DataBind();
         }
      }
   }
}
```

->DataReader vs DataSet
Dataset:

- DataSet object can contain multiple rowsets from the same data source as well as from the relationships between them.
- Dataset is a disconnected architecture
- Dataset can persist data.

Datareader:

- DataReader provides forward-only and read-only access to data.
- Datareader is connected architecture
- Datareader can not persist data.

->ADO vs ADO.Net,[Difference between ado and ado.net]

ADO	ADO.NET
ADO use connected data environment	ADO.net use disconnected data environment /Architecture.

ADO used OLE DB to access data and isCOM-based	ADO.net uses XML as the format for transmitting data to and from your databaseand web application
ADO, Record set, is like a single table or queryresult	ADO.net Dataset, can contain multiple tablesfrom any data source
ADO is sometime problematic because firewall prohibits many types of request	ADO.net there is no such problem because XML is completely firewall-proof
It has a limited number of data types	it support large and rich datatypes
ADO allows you to create client side cursorsonly	ADO.NET gives you the choice of either usingclient side or server side cursors
In ADO, we cannot send multiple transactions using a single connection instance.	In ADO.Net we can send multiple transactions using a single connection instance
Limited XML Support	Robust XML Support
ADO objects communicate in binary mode.	ADO.NET uses XML for passing the data.

Advantages/benefits of ADO.Net over ADO

1. ADO.NET Does Not Depend On Continuously Live Connections
2. Database Interactions Are Performed Using Data Commands
3. Data Can Be Cached in Datasets
4. Datasets Are Independent of Data Sources
5. Data Is Persisted as XML
6. Schemas Define Data Structures
7. Interoperability
8. Maintainability
9. Programmability
10. Performance

->Data Binding - Single value Data Binding:

->Allows you to take a Page variable, property, or an expression and insert it dynamically into a page.

-> In other words, it allows you to pop data into HTML elements.

->To implement single-value Data Binding, use a Binding expression like <%# Var_Prop_Expr %>
within your .aspx file.

->Then, convert all the data binding expressions on the Page via the DataBind() method of the Page object.

Example

ASPX

```
<!-- We could have used %= instead of %#, and if so then we would not have to call DataBind() -->
There were <%# TransactionCount %> transactions today. <br /><br />
I see that you are using <%# Request.Browser.Browser %>.
```

Code-Behind:

```
public partial class SimpleDataBinding : System.Web.UI.Page
{
        protected int TransactionCount;
        protected void Page_Load(object sender, EventArgs e)
        {
```

```
            // You could use database code here to look up a value for TransactionCount.
            TransactionCount = 10;
            // Now convert all the data binding expressions on the page.
            this.DataBind();
        }
}
```

-> Repeated Value Data Binding,

->Bind data to List controls (e.g. ListBox) or Rich Data controls (e.g. GridView), which are all controls with a DataSource property.

->To use Repeated-Value Data Binding, create / fill / retrieve your data object (i.e. SqlDataReader or DataSet) and then assign it as your control's DataSource (often done in the Page.Load event or in the aspx file).

->When you call DataBind(), the control automatically creates a full list using all the corresponding values.

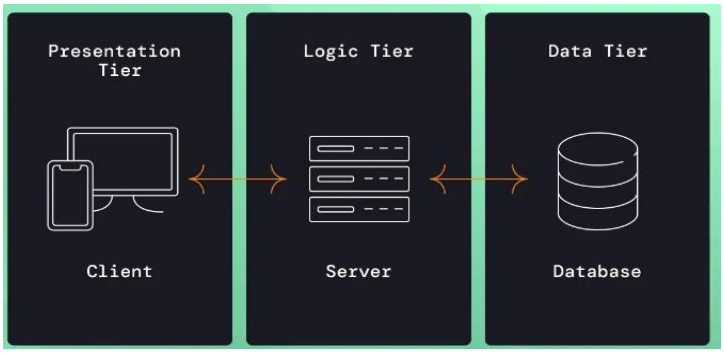
Example

You can bind the same data list object to multiple different controls. Consider the following example, which compares all the types of list controls at your disposal by loading them with the same information:

```
Protected Sub Page_Load(ByVal sender As Object,  ByVal e As EventArgs) Handles Me.Load

   ' Create and fill the collection.
   Dim fruit As New List(Of String)()
   fruit.Add("Kiwi")
   fruit.Add("Pear")
   fruit.Add("Mango")
   fruit.Add("Blueberry")
   fruit.Add("Apricot")
   fruit.Add("Banana")
   fruit.Add("Peach")
   fruit.Add("Plum")

   ' Define the binding for the list controls.
   MyListBox.DataSource = fruit
   MyDropDownListBox.DataSource = fruit
   MyHtmlSelect.DataSource = fruit
   MyCheckBoxList.DataSource = fruit
   MyRadioButtonList.DataSource = fruit

   ' Activate the binding.
   Me.DataBind()
End Sub
```

->Three tiers of Web Application

The Three-Tier Client-Server Architecture in distributed systems is a design model that separates applications into three distinct layers:



1. Presentation tier

Focus: User interaction and display of information.

Role: This is the interface that users see and interact with. It gathers input, formats and sanitizes data, and displays the results returned from the other tiers.

Technologies:

Web Development: HTML, CSS/SCSS/Sass, TypeScript/JavaScript, front-end frameworks (React, Angular, Vue.js), a web server.

Mobile Development: Platform-specific technologies (Swift, Kotlin, etc.).

Desktop Applications: Platform-specific UI libraries or third-party cross-platform development tools.

2. Logic tier

Focus: Core functionality and business logic.

Role: This tier is the brain of the application. It processes data, implements business rules and logic, further validates input, and coordinates interactions between the presentation and data tiers.

Technologies:

Programming Languages: Java, Python, JavaScript, C#, Ruby, etc.

Web Frameworks: Spring, Django, Ruby on Rails, etc.

App Server/Web Server

3. Data tier

Focus: Persistent storage and management of data.

Role: This tier reliably stores the application's data and handles all access requests. It protects data integrity and ensures consistency.

Technologies:

Database servers: Relational (MySQL, PostgreSQL, Microsoft SQL Server) or NoSQL (MongoDB, Cassandra).

Database Management Systems: Provide tools to create, access, and manage data.

Storage providers (AWS S3, Azure Blobs, etc)

Examples

E-commerce websites

Presentation Layer: The online storefront with product catalogs, shopping carts, and checkout interfaces.

Logic Layer: Handles searching, order processing, inventory management, interfacing with 3rd-party payment vendors, and business rules like discounts and promotions.

Data Layer: Stores product information, customer data, order history, and financial transactions in a database.

Content management systems (CMS)

Presentation Layer: The administrative dashboard and the public-facing website.

Logic Layer: Manages content creation, editing, publishing, and the website's structure and logic based on rules, permissions, schedules, and configuration

Data Layer: Stores articles, media files, user information, and website settings.